

数据库系统原理

胡 敏 教授 博士生导师
合肥工业大学 计算机与信息学院
jsjxhumin@hfut.edu.cn

第4章 数据库安全性



厚德 笃学 崇实 尚新

第4章 数据库安全性

■ 本章目标

- 掌握什么是数据库的安全性问题
- 牢固掌握数据库管理系统实现数据库安全性控制的常用方法和技术

■ 本章重点

- 使用GRANT 语句和 REVOKE 语句实现自主存取控制功能
- 使用CREATE ROLE语句创建角色，用GRANT 语句给角色授权
- 掌握视图机制在数据库安全保护中的作用

■ 本章难点

- 强制存取控制机制中确定主体能否存取客体的存取规则
- 要理解并掌握存取规则为什么要这样规定



第4章 数据库安全性

问题的提出:数据共享与安全的矛盾



核心原则：共享需受控

数据库价值在于共享，但必须是有条件、受控制的开放，而非无底线暴露。



现实风险：攻击的靶心

集中存储的海量敏感数据如同“磁铁”，极易成为黑客攻击的主要目标。



国家与机构机密

国家机密、军事秘密等高阶数据



企业战略资产

实验数据、营销策略、销售计划等



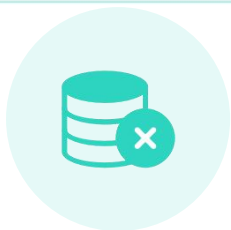
个人隐私信息

客户档案、医疗记录、银行储蓄等

安全是数据共享的前提，失去安全的共享毫无价值



数据库安全的严峻性：数据泄露的重灾区



78% +

IDC报告指出，绝大部分敏感数据集中存储于数据库中，是泄露重灾区

存储核心风险点



98%

Verizon报告显示，几乎所有敏感数据窃取行为均直接或间接指向数据库服务器

攻击首选目标



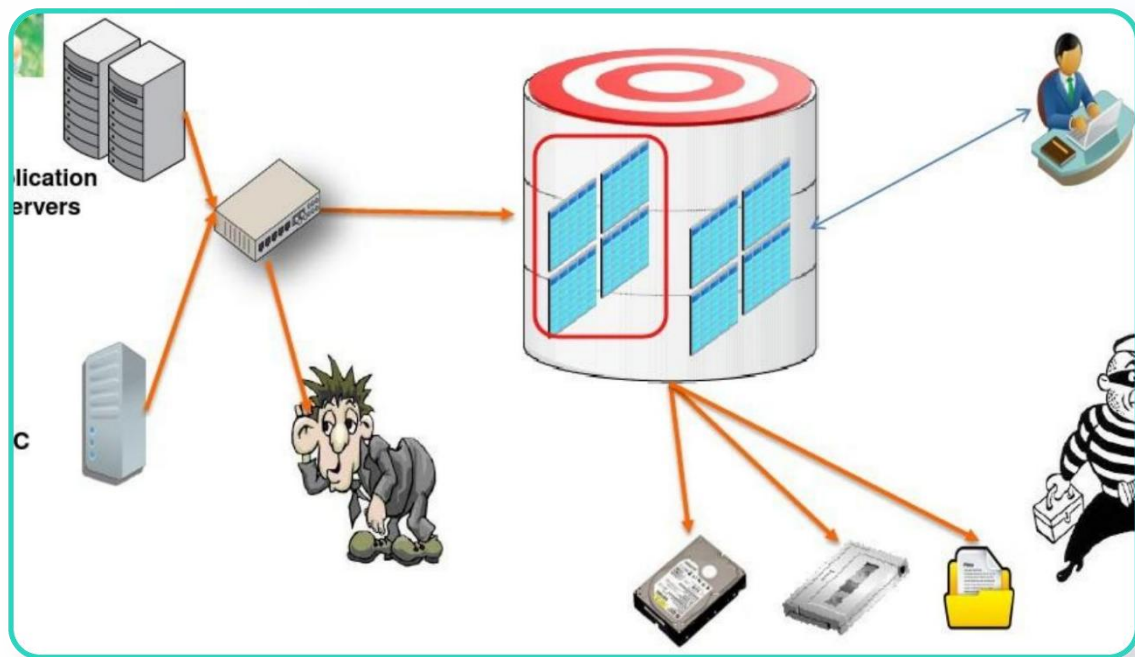
核心防护环节

数据库是信息资产的“金库”，是企业信息安全体系中最关键、最需要加固的防线

安全建设重中之重

结论：忽视数据库安全，就等同于将企业核心资产直接暴露在风险之中

攻击路径示例：多层次的威胁



外部攻击：应用层渗透

利用互联网漏洞突破应用服务器，进而窃取数据



内部威胁：权限滥用

内部员工违规访问或拷贝核心数据，危害更隐蔽



旁路攻击：直接突破

绕过应用层防护，直接对数据库文件发起攻击

图示：攻击者通过多维度路径渗透数据库的典型场景



核心启示：单一防护易被突破，必须构建多层次、全方位的纵深防御体系



第4章 数据库安全性

4.1 数据库安全性概述

4.2 数据库安全性控制

4.3 视图机制

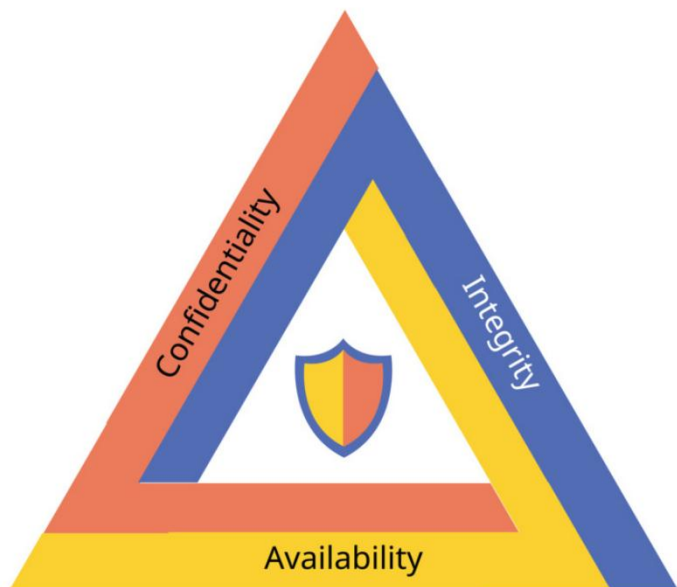
4.4 审计 (Audit)

4.5 数据加密

4.6 小结



4.1 数据库安全性概述



CIA 模型：信息安全的核心基石



计算机系统安全性

保护硬件、软件和数据，防止因偶然或恶意原因导致的破坏、更改或泄露。



数据库安全性

在DBMS控制下，确保只有合法权限的用户才能访问其被授权存取的数据。



保密性 (C)

不泄露给未授权者



完整性 (I)

防止非法篡改



可用性 (A)

需要时正常访问



4.1 数据库安全性概述

■ 计算机系统安全性问题可以分成三大类：

□ 技术安全类

□ 管理安全类

□ 政策法律类

软硬件意外故障、场地的意外事故、管理不善

政府部门建立的有关计算机犯罪、数据安全保密的法律道德准则和政策法规、法令。

路。



4.1.1 数据库不安全因素

数据库的不安全因素与防范对策

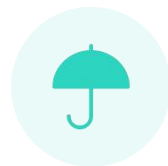


⚠️ 核心威胁：外部入侵 / 数据泄露 / 环境漏洞



非授权用户的恶意存取

手段：黑客窃取凭证假冒合法用户 | 防范：身份鉴别、存取控制



重要敏感数据泄露

手段：攻击者直接盗窃机密信息 | 防范：MAC控制、数据加密、审计



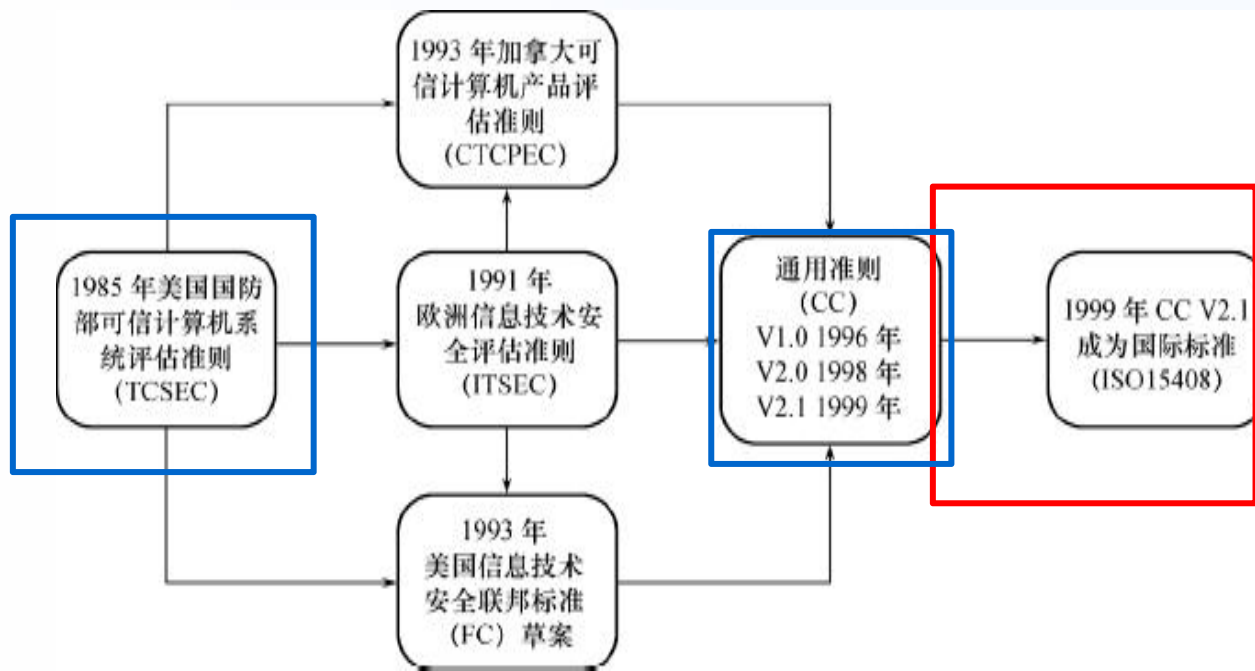
安全环境的脆弱性

关联：依赖OS与网络安全 | 防范：数据库防火墙、堡垒机

全方位构建可信数据库安全体系，守护数据核心资产

4.1.2 安全标准简介

- **TCSEC**(Trust Computer System Evaluation Criteria)标准
- **CC**(Trusted Database Interpretation)标准



图示：信息安全标准从TCSEC到CC的演进历程

安全标准的演进之路

- **TCSEC (桔皮书)**：1985年发布，开创计算机安全评估先河。
- **TDI (紫皮书)**：1991年发布，将标准扩展至数据库领域。
- **CC (通用准则)**：ISO 15408国际标准，当前主流安全标准。

核心思想转变

从单一关注系统功能，转向兼顾**安全功能要求 (FR)**与**安全保证要求 (AR)**。



4.1.2 安全标准简介

■ TCSEC/TDI 安全级别划分体系



D级 • 最低保护

无安全保护机制
仅保证系统运行
无任何安全性保障



C级 • 自主/受控

C1: 自主存取控制 (DAC)
C2: 增加审计与隔离
商业安全产品最低门槛



B级 • 强制/结构化

B1: 强制存取控制 (MAC)
B2: 结构化保护, 形式化的
安全策略模型。DAC+MAC
B3: 安全域划分, 监控器+审
计跟踪。



A1级 • 验证设计

最高安全级别
形式化最高级设计与验证
完全的访问控制与审计追踪

核心逻辑：从D级无保护到A1级形式化验证，安全强度逐级递增，控制粒度逐级变细

4.1.2 安全标准简介

■ CC 评估保证级 (EAL) 与安全标准解析



CC标准核心要素

- 安全功能要求：产品应具备的核心防护能力
- 安全保证要求：确保功能有效落地的验证措施



评估保证级 (EAL)

衡量安全保证程度的标尺，从EAL1到EAL7，数字越高代表验证越严格，安全性保证越强。

EAL 与 TCSEC 等级映射



EAL2 $\approx$ C1



EAL3 $\approx$ C2



EAL4 $\approx$ B1



EAL5 $\approx$ B2



EAL6 $\approx$ B3



EAL7 $\approx$ A1

价值：为产品安全性提供全球通用的量化评估标尺



第4章 数据库的安全性

4.1 计算机安全性概述

4.2 数据库安全性控制

4.3 视图机制

4.4 审计 (Audit)

4.5 数据加密

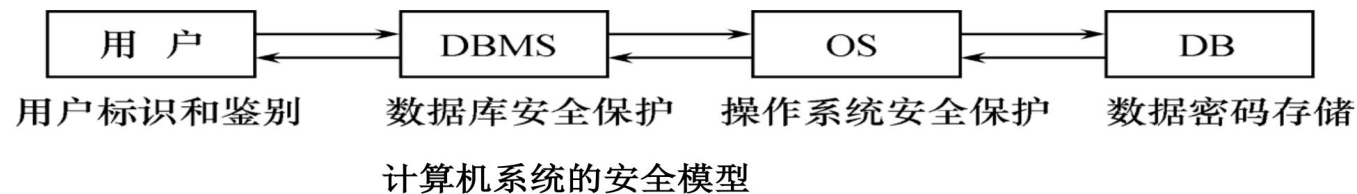
4.6 小结

- ①用户标识和鉴定
- ②存取控制
- ③自主存取控制方法
- ④授权与回收
- ⑤数据库角色
- ⑥强制存取控制方法



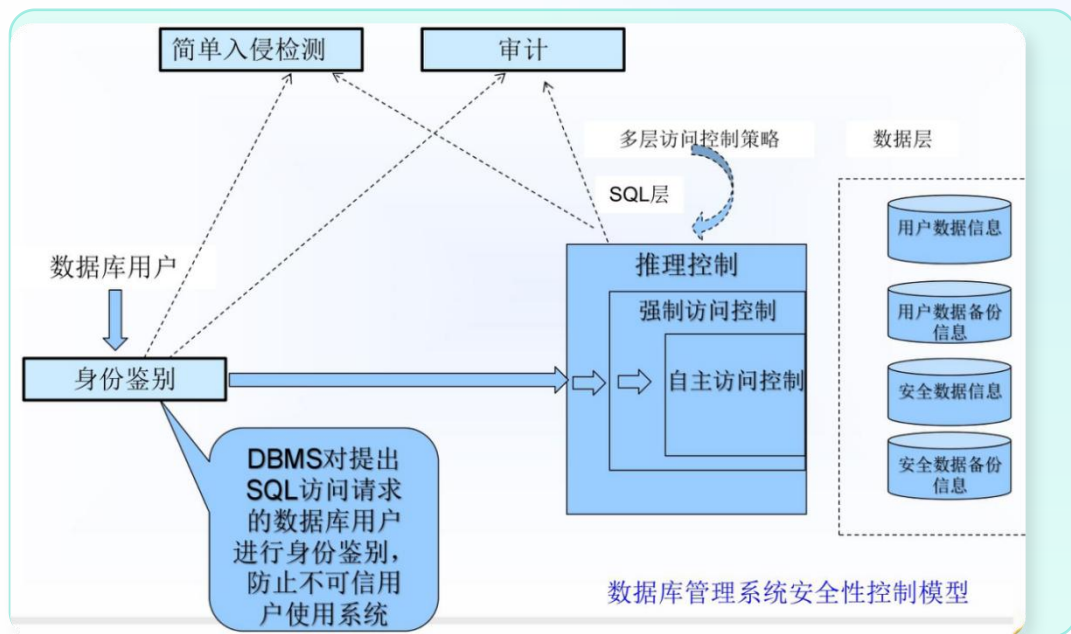
4.2 数据库安全性控制

■ 数据库安全控制模型：多层防护体系



高

低



图示：数据库管理系统安全性控制架构全貌



用户层：严格的用户标识与身份鉴别机制



DBMS层：核心存取控制与SQL层安全保护



OS层：操作系统级别的底层环境安全防护



数据层：存储数据的加密保护与完整性校验

核心机制：DBMS存取控制直接决定用户身份与操作权限的最终放行



4.2 数据库安全性控制（续）

■ DBMS的安全机制

➤ 自主安全性机制：存取控制(Access Control)

通过权限在用户之间的传递，使用户自主管理数据库安全性

➤ 强制安全性机制：

通过对数据和用户强制分类，使得不同类别用户能够访问不同类别的数据

➤ 推断控制机制：(可参阅相关文献)

✓ 防止通过历史信息，推断出不该被其知道的信息；

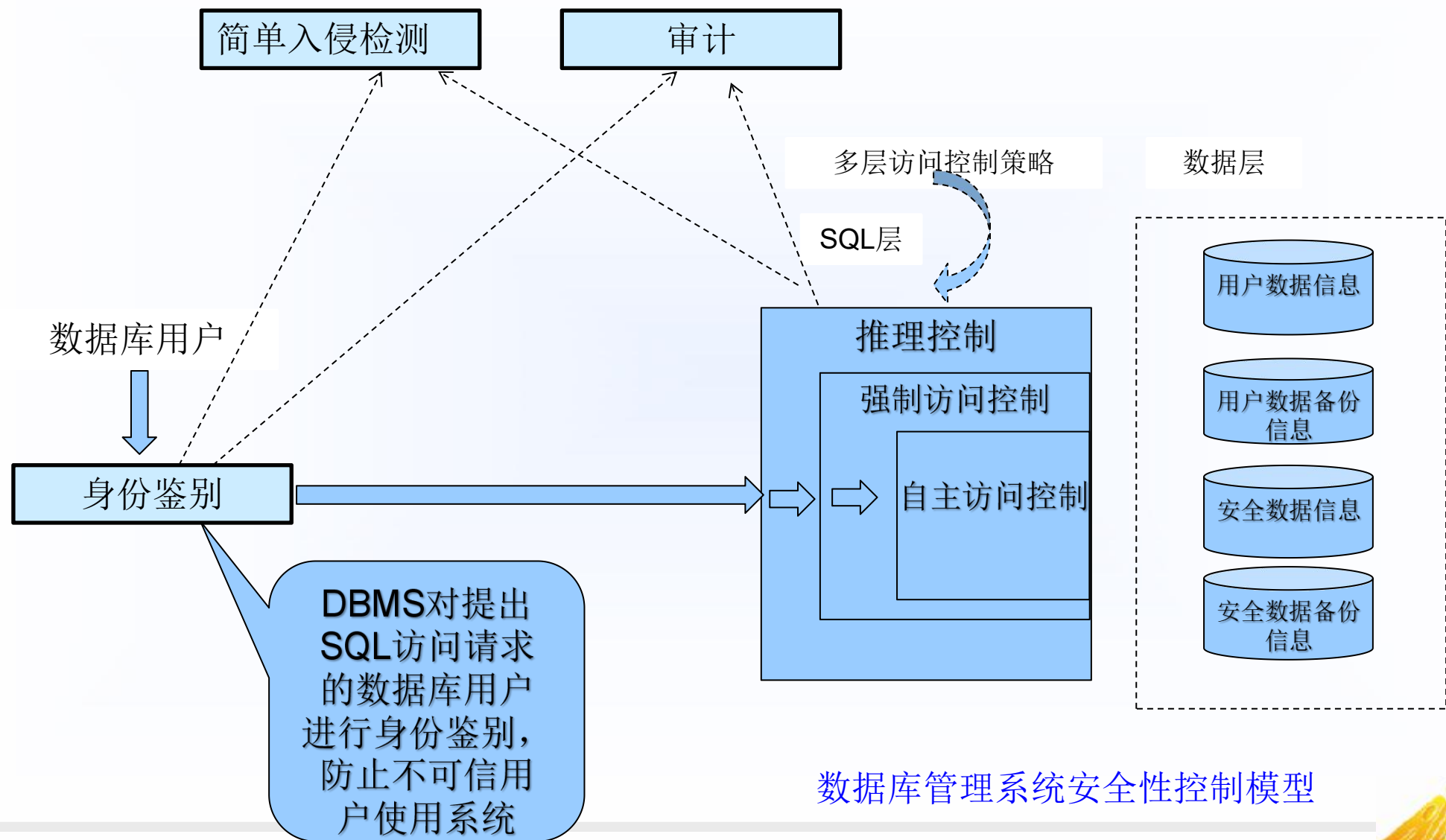
✓ 防止通过公开信息(通常是一些聚集信息)推断出私密信息(个体信息)，通常在一些由个体数据构成的公共数据库中此问题尤为重要

➤ 数据加密存储机制：(可参阅相关文献)

通过加密、解密保护数据，密钥、加密/解密方法与传输。



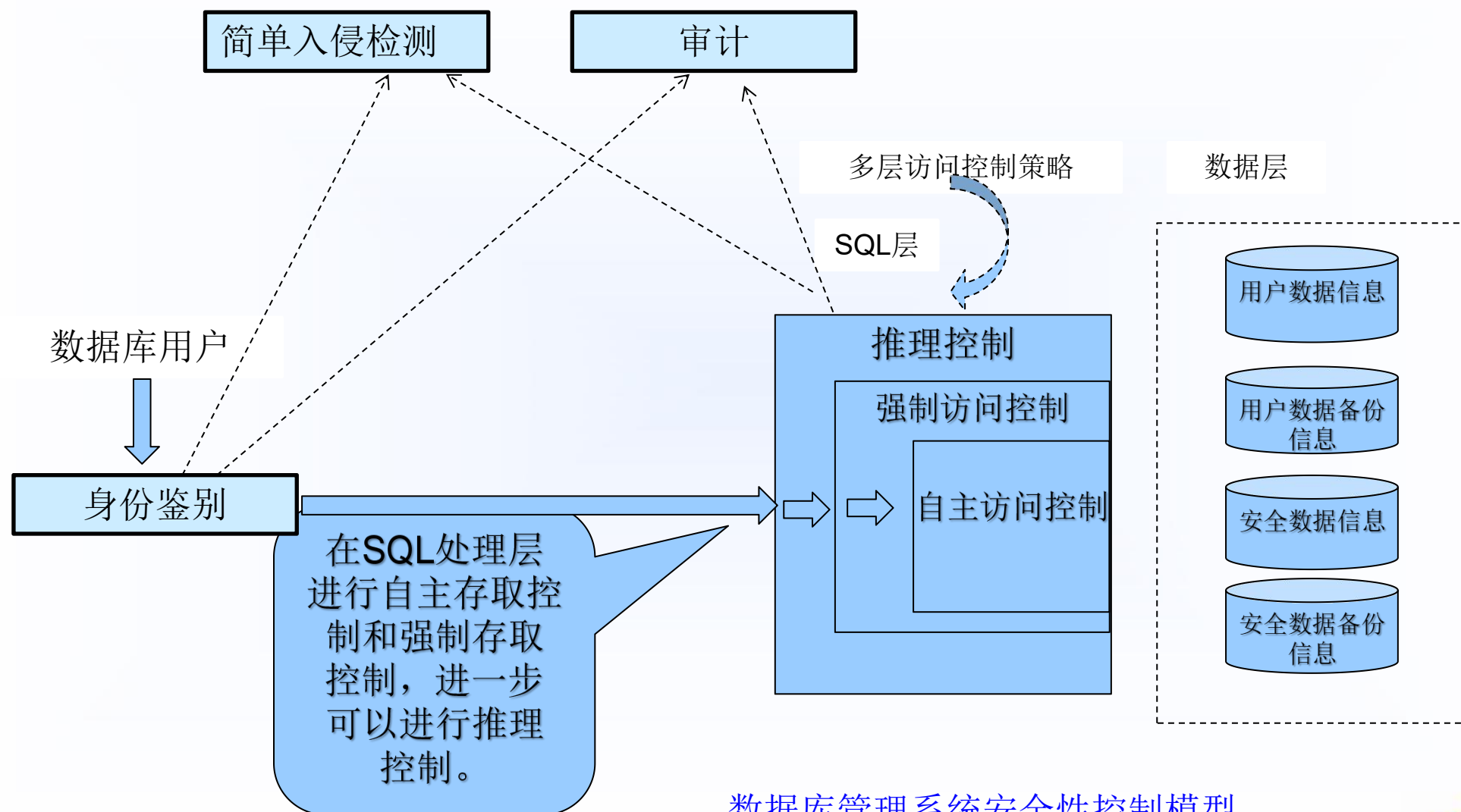
数据库安全性控制（续）



数据库管理系统安全性控制模型



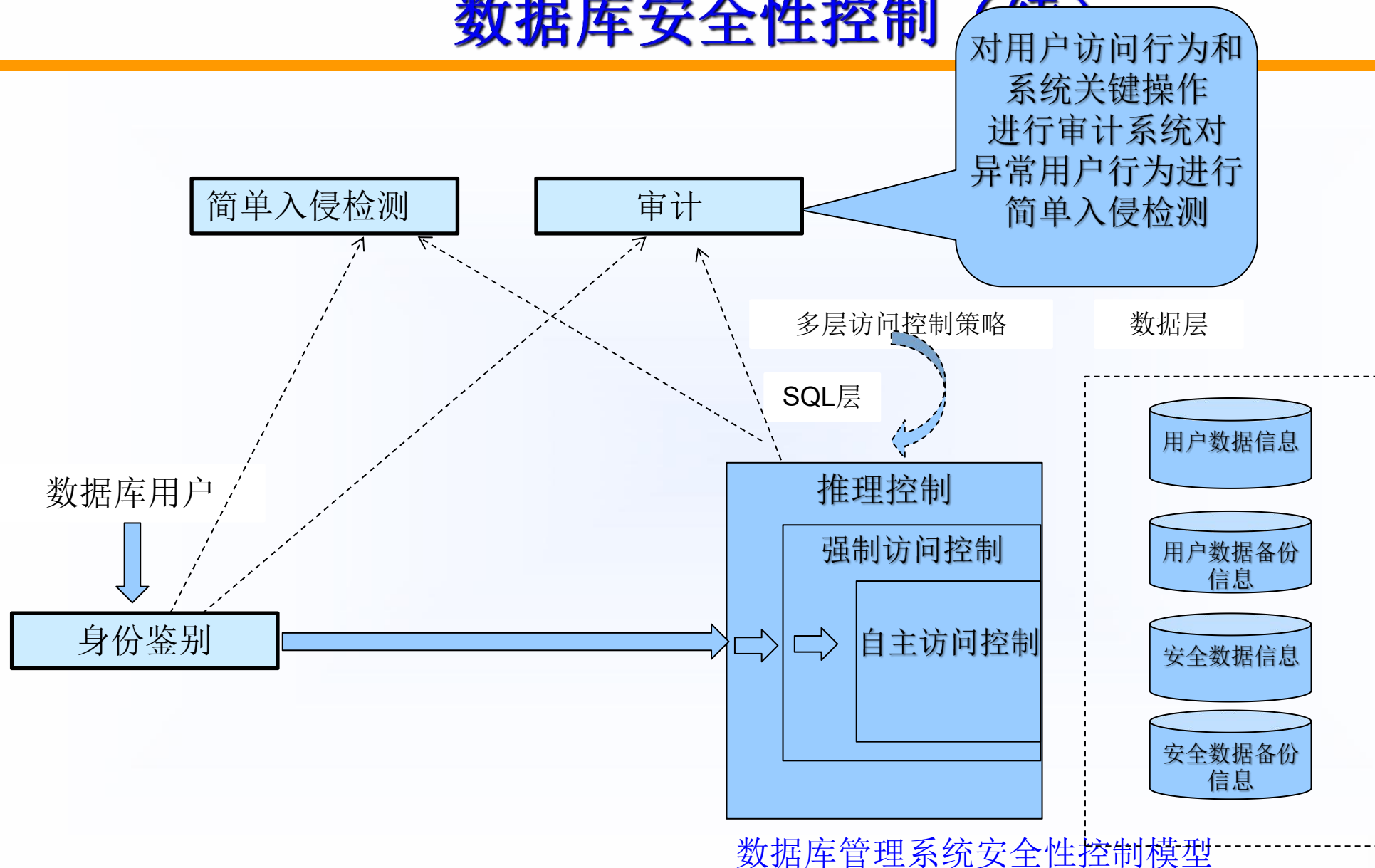
数据库安全性控制（续）



数据库管理系统安全性控制模型



数据库安全性控制 (续)



4.2 数据库安全性控制

4.2.1 用户标识与鉴别(Identification & Authentication)

4.2.2 存取控制

4.2.3 自主存取控制(DAC)方法

4.2.4 授权(Authorization)与回收

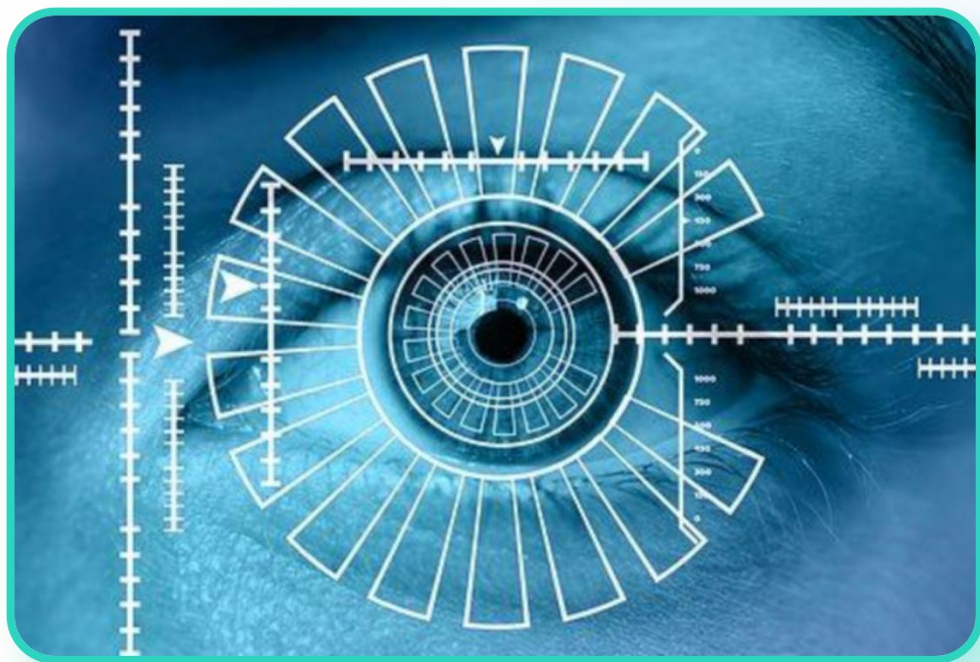
4.2.5 数据库角色

4.2.6 强制存取控制(MAC)方法



4.2.1 用户标识和鉴别

用户标识与鉴别：数据库安全的第一道防线



技术示意：虹膜识别（生物特征鉴别）

基于人体生物特征的高精度身份验证手段



核心定义

通过用户名和口令的方法鉴别用户身份。系统提供的最外层安全保护措施，用于精准识别“你是谁”并验证身份真实性。



静态口令

最基础的验证方式，便捷但易被窃取。



动态口令

基于挑战-应答机制，口令实时动态变化。



生物特征

利用指纹、虹膜等唯一性特征进行鉴别。



智能卡鉴别

结合硬件设备（如U盾）与密码双重验证。



4.2.2 存取控制

- 存取控制机制：数据库安全的核心防线。
- 确保只授权给有资格的用户访问数据库的权限，同时令所有未授权的人员无法接近数据，这主要通过数据库系统的存取控制机制实现。



01 定义用户权限

通过SQL的GRANT/REVOKE语句定义，并存储在数据字典中。



02 合法权限检查

用户操作时，DBMS自动查询字典进行权限核验。



DAC 自主存取控制

基于身份，用户可自主转让权限。



MAC 强制存取控制

基于密级与许可证，安全级别更高。

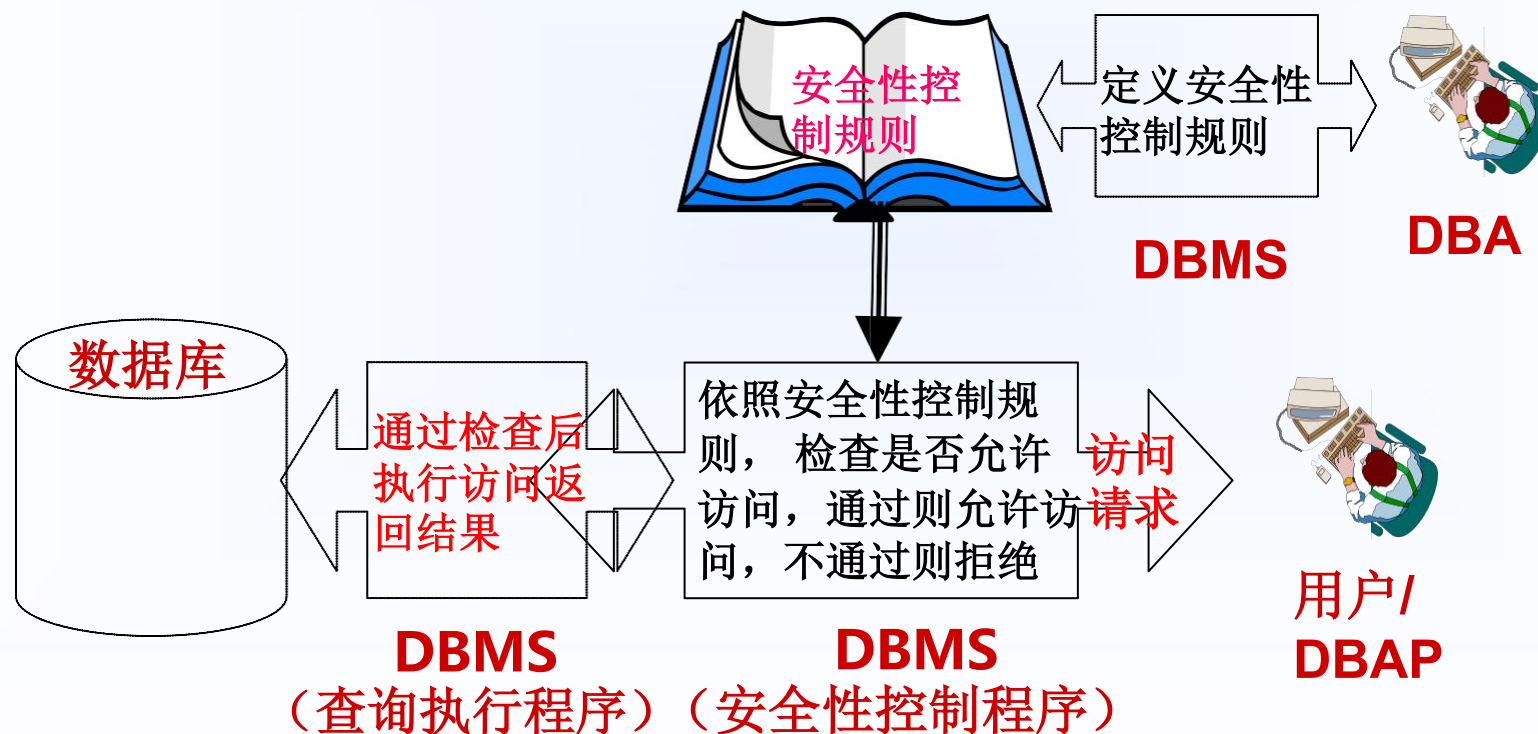
- 用户权限定义和合法权检查机制一起组成了**DBMS的安全子系统**



4.2.3 自主存取控制

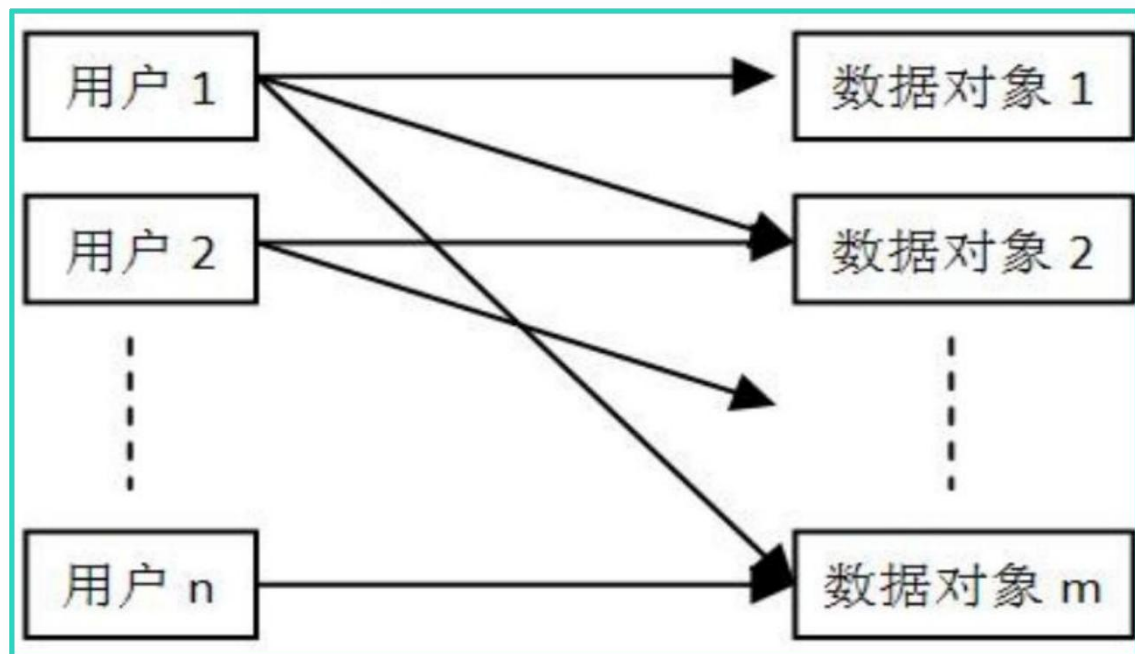
■ 自主访问控制方法

- DBMS允许用户定义一些安全性控制规则(用SQL-DCL来定义)
- 当有DB访问操作时，DBMS自动按照安全性控制规则进行检查，检查通过则允许访问，不通过则不允许访问。



4.2.3 自主存取控制方法

■ 自主存取控制（Discretionary Access Control, 简称DAC），C2级，灵活



图示：用户与数据对象的多对多权限映射关系



核心思想：自主授权

用户拥有“自主”处置权，可将自身权限授予他人或随时收回。



权限关系：多对多映射

单用户可操作多个数据对象，多用户也可共享同一对象的不同权限。



技术实现：SQL 核心语句

通过 **GRANT** 语句进行权限授予，使用 **REVOKE** 语句进行权限回收。



4.2.3 自主存取控制

- SQL语言包含了DDL, DML和DCL。数据库安全性控制是属于DCL范畴
- 授权机制---自主安全性；视图的运用
- 关系级别(普通用户) ← 账户级别(程序员用户) ← 超级用户(DBA)

□ (级别1)Select 读(读DB, Table, Record, Attribute, ...)

□ (级别2)Modify: 更新

✓ Insert : 插入

✓ Update : 更新

✓ Delete : 删除

□ (级别3)Create: 创建(创建表空间、模式、表、索引、视图等)

✓ Create : 创建

✓ Alter : 更新

✓ Drop : 删除

- 级别高的权利自动包含级别低的权利。如某人拥有更新的权利，它也自动 拥有读的权利。在有些DBMS中，将级别3的权利称为账户级别的权利，而将级别1和2称为关系级别的权利。



4.2.3 自主存取控制方法

■ 定义存取权限要素

- ✓ 数据对象
- ✓ 操作类型

■ 关系数据库系统中的存取权限类型

对象数据	对象	
数据库	模式	CREATE DATA
	基本表	CREATE
模式	视图	C
	索引	CF
数据	基本表和视图	SELECT,INSERT,UP AL
	属性列	SELECT,INSERT, F

选择页

常规

权限

更改跟踪

存储

扩展属性

表属性 - Td_zzmmdm

脚本 帮助

架构(S): dbo

查看架构权限

表名(N): Td_zzmmdm

用户或角色(U):

搜索

名称	类型
guest	用户

guest 的权限(P):

列权限(C)...

显式

有效

权限	授权者	授予	具有授予...	拒绝
插入	dbo	☒	☒	
查看定义	dbo	☒	☐	
查看更改跟踪	dbo	☒	☐	
更改	dbo	☐	☐	
更新	dbo	☒	☐	
接管所有权	dbo	☐	☐	
拒绝	dbo	☐	☐	

连接

服务器: DESKTOP-U2BTSAG\SQLEXPRESS

连接: DESKTOP-U2BTSAG\humin

查看连接属性

进度

已成功完成脚本操作。



4.2.3 用户权限设置

✓ 关系数据库系统中访问控制对象

对象类型	对象	操作类型
数据库	模式	CREATE SCHEMA
	基本表	CREATE TABLE, ALTER TABLE
模式	视图	CREATE VIEW
	索引	CREATE INDEX
数据	基本表和视图	SELECT, INSERT, UPDATE, DELETE, REFERENCES, ALL PRIVILEGES
数据	属性列	SELECT, INSERT, UPDATE, REFERENCES, ALL PRIVILEGES



4.2.3 自主存取控制方法

■ 关系系统中的存取权限定义方法

GRANT/REVOKE

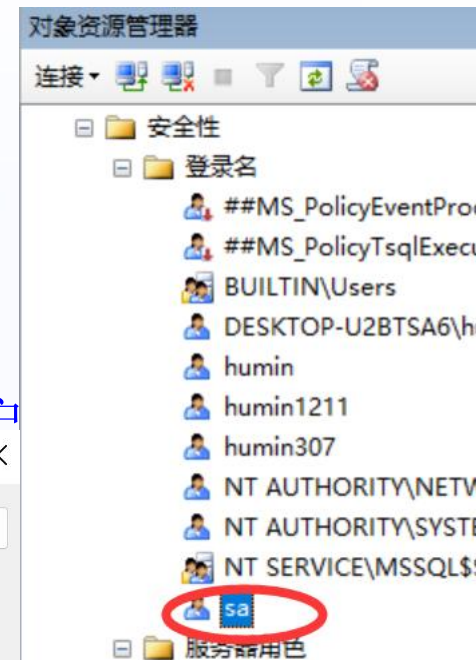
■ DBMS专门提供一个DBA账户

该账户是一个超级用户或称系统用户。DBA利用该账户的特权可以进行用户管理

予和撤消、安全

■ DBMS实现数

1. 用户或DBA
2. SQL的GRAN
3. DBMS把授
4. 当用户提出



灵授

行操作请求



4.2.4 授权(Authorization)与回收

■ 授权权限GRANT

一般格式:

GRANT <权限>[, <权限>]...

[ON <对象名>]

TO <用户>[, <用户>].

[WITH GRANT OPTION];

指定该子句: 可以再转授权限
没有指定: 不能传播权限
具有授权的权限

谁定义? DBA和表的建立者 (即表的属主)

GRANT功能: 将对指定操作对象的指定操作权限授予指定的用户。



4.2.4 授权与回收

■ 谁可以发出GRANT:

➤ DBA、对象创建者（Owner）、拥有该权限的用户。

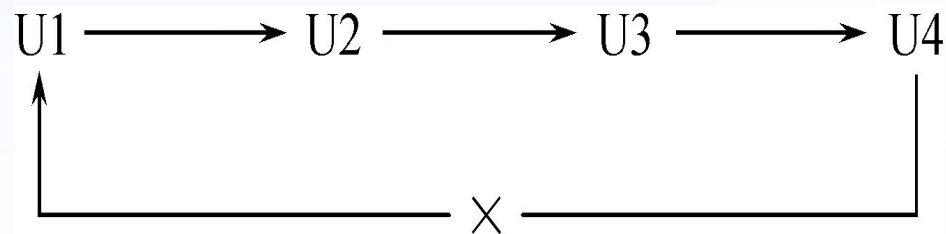
■ 可接受权限的用户：

➤ 一个或多个具体用户、PUBLIC（全体用户）。

■ WITH GRANT OPTION子句

获得某种权限的用户还可以把这种权限再授予别的用户

□ 不允许循环授权:



4.2.4 授权与回收

■ 安全性设置实例：

授权过程

1. 创建新用户u1,u2,u3,u4,u5
2. 授予用户u1,u2,u3,u4,u5权限connect
3. 授予用户特定的对象权限



4.2.4 授权与回收

[例1] 将查询Students表的select权限授给用户U1。

```
GRANT SELECT ON table Students TO U1;
```

[例2] 将对Students表和Course表的全部权限授予用户U2和U3。

```
GRANT ALL PRIVILEGES ON TABLE Students, Course TO U2, U3;
```

[例3] 将对表SC的查询权限授予所有用户。

```
GRANT SELECT ON TABLE SC TO PUBLIC;
```

[例4] 将查询Students表和修改学生学号的权限授给用户U4。

```
GRANT UPDATE(Sno), SELECT ON TABLE Students TO U4;
```

[例5] 将SC的INSERT权限授予U5，并允许他将此权限授予其他用户。

```
GRANT INSERT ON TABLE SC TO U5 WITH GRANT OPTION;
```

U5还可以将此权限授予U6，。。。



传播权限

执行例5后，U5不仅拥有了对表SC的INSERT权限，还可以传播此权限：*U5还可以将此权限授予U6，。。。*

[例6] GRANT INSERT
 ON TABLE SC TO U6 WITH GRANT OPTION;

[例7] 同样U6还可以将此权限授予U7。
 GRANT INSERT ON TABLE SC TO U7;;

但U7不能再传播此权限。;



4.2.4 授权与回收

■ 撤销 (收回) 权限

授予的权限可以由DBA或其他授权者用REVOKE语句收回

➤ REVOKE语句的一般格式为:

REVOKE <权限>[, <权限>]...

[ON <对象类型> <对象名>]

FROM <用户>[, <用户>]...

[**CASCADE** | **RESTRICT**]

不同DBMS的缺省值
不相同



4.2.4 授权与回收

[例8] 把用户U4修改学生学号的权限收回。

```
REVOKE UPDATE(Sno) ON TABLE Students FROM U4;
```

[例9] 收回所有用户对表SC的查询权限。

```
REVOKE SELECT ON TABLE SC FROM PUBLIC;
```

[例10] 把用户U5对SC表的INSERT权限收回。

```
REVOKE INSERT ON TABLE SC FROM U5 CASCADE ;
```

- 如果系统缺省值为RESTRICT，回收U5的INSERT权限时应该使用CASCADE短语，否则拒绝执行该语句
- 如果U6或U7还从其他用户处获得对SC表的INSERT权限，则他们仍具有此权限，系统只收回直接或间接从U5处获得的权限



4.2.4 授权与回收

■ 创建数据库模式的权限

- **DBA**在创建用户时实现！

```
CREATE USER <username>  
[WITH] [DBA | RESOURCE | CONNECT]
```

不是SQL标准，各DBMS产品会有所不同

拥有的权限	可否执行的操作			
	CREATE USER	CREATE SCHEMA	CREATE TABLE	登录数据库 执行数据查询 和操纵
DBA	可以	可以	可以	可以
RESOURCE	不可以	不可以	可以	可以
CONNECT	不可以	不可以	不可以	可以，但必须拥有相应权限



4.2.4 授权与回收

Oracle 授权机制:

■ 创建用户

create user <用户名> identified <by 口令>;

■ 授予用户系统权限

grant create table, create view, create synonym,
create sequence, create trigger, create index, create
procedure to <用户|角色> [with admin option];

■ 授予用户对象权限

grant select,delete,update,insert on <表名>
to 用户|角色|public [with grant option];

MySQL 创建用户

■ CREATE USER [IDENTIFIED BY [PASSWORD] 'password'] [,用户 [IDENTIFIED BY [PASSWORD] 'password']]



4.2.5 数据库角色

数据库角色的引入

■ 例:

```
GRANT SELECT, UPDATE, INSERT  
  ON TABLE Student  
  TO U1, U2;  
GRANT SELECT, UPDATE(Ccredit)  
  ON TABLE Course  
  TO U1, U2;  
GRANT SELECT, INSERT  
  ON TABLE SC  
  TO U1, U2;
```

```
GRANT SELECT, UPDATE, INSERT  
  ON TABLE Student  
  TO U3;  
GRANT SELECT, UPDATE(Ccredit)  
  ON TABLE Course  
  TO U3;  
GRANT SELECT, INSERT  
  ON TABLE SC  
  TO U3;
```



数据库角色的引入

■ 例:

```
GRANT SELECT, UPDATE, INSERT  
ON TABLE Student  
TO U4;  
GRANT SELECT, UPDATE(Ccredit)  
ON TABLE Course  
TO U4;  
GRANT SELECT, INSERT  
ON TABLE SC  
TO U4;
```

```
GRANT SELECT, UPDATE, INSERT  
ON TABLE Student  
TO U5;  
GRANT SELECT, UPDATE(Ccredit)  
ON TABLE Course  
TO U5;  
GRANT SELECT, INSERT  
ON TABLE SC  
TO U5;
```



简化授权过程。



4.2.5 数据库角色

■ 基于角色的访问控制**RBAC**(Role-Based Access Control)

- ✓ 数据库角色：被命名的一组与数据库操作相关的权限。
- ✓ 角色是权限的集合。
- ✓ 可以为一组具有相同权限的用户创建一个角色。
- ✓ 简化授权的过程。

■ 角色的创建： **CREATE ROLE** <角色名>;

■ 给角色授权：

GRANT <权限> [, <权限>] ... **ON** <对象类型> 对象名
TO <角色> [, <角色>] ...;



使用角色管理数据库权限

- 将一个角色授予其他的角色或用户

GRANT <角色1>[,<角色2>]...

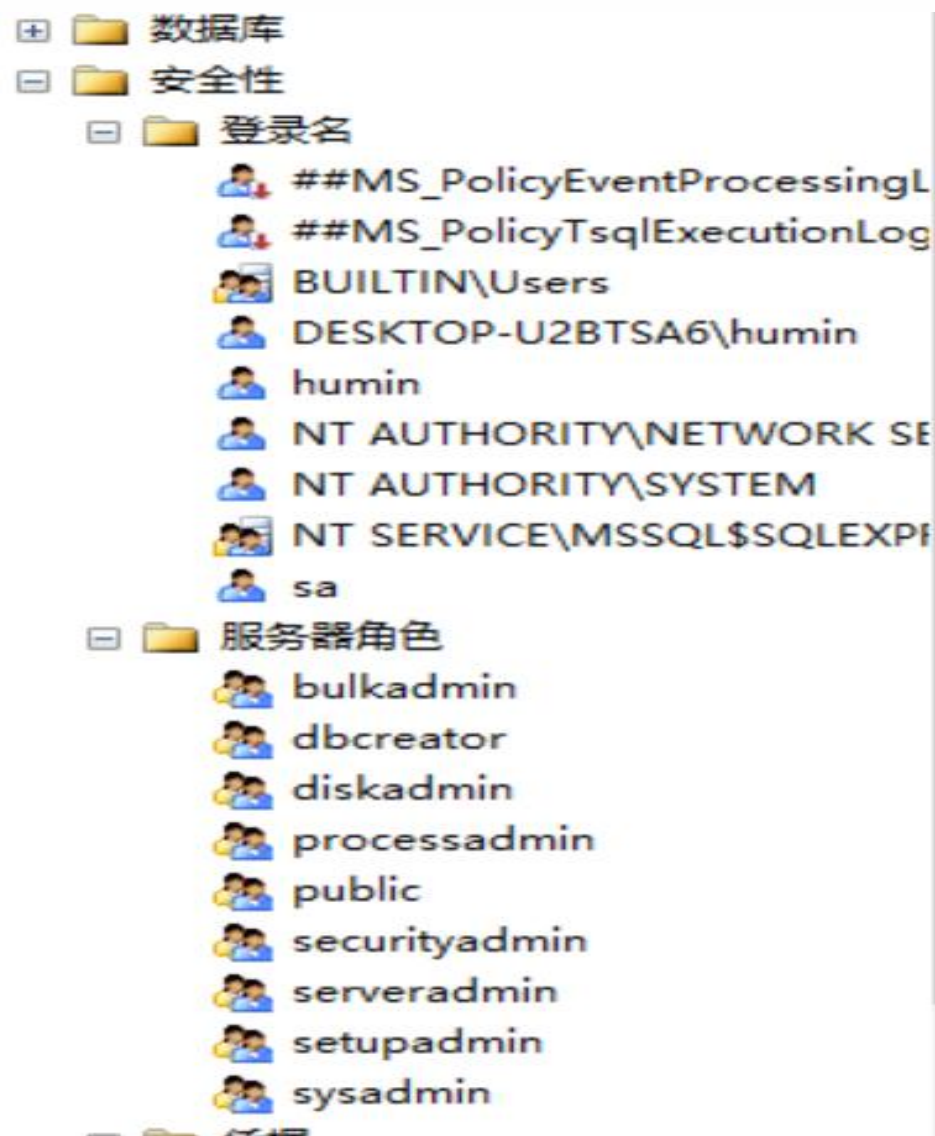
TO <角色3>[,<用户1>]...

[WITH ADMIN OPTION]

指定WITH ADMIN OPTION，
则获得权限的角色或用户还可以把这种权限授予其他角色

- 一个角色的权限：直接授予这个角色的全部权限加上其他角
- 授予者是角色的创建者或拥有在这个角色上的ADMIN OPTION





使用角色管理数据库权限（续）

■ 角色权限的收回：

REVOKE <权限> [, <权限>] ... **ON** <对象类型> <对象名>
FROM <角色> [, <角色>] ...;

➤ 用户可以回收角色的权限，从而修改角色拥有的权限

➤ **REVOKE**执行者是

- ✓ 角色的创建者
- ✓ 拥有在这个（些）角色上的**ADMIN OPTION**



例题

[例11] 通过角色来实现权限管理。

步骤如下：

(1) 首先创建一个角色 R1

CREATE ROLE R1;

(2) 然后使用GRANT语句，使角色R1拥有Student表的SELECT、UPDATE、INSERT权限

GRANT SELECT, UPDATE, INSERT

ON TABLE Student

TO R1;

(3) 将这个角色授予王平，张明，赵玲。使他们具有角色R1所包含的全部权限

GRANT R1

TO 王平,张明,赵玲;

(4) 可以一次性通过R1来回收王平的这3个权限

REVOKE R1

FROM 王平;



例题（续）

[例12] 增加角色的权限

```
GRANT DELETE  
ON TABLE Student  
TO R1;
```

使角色**R1**在原来的基础上增加了**Student**表的**DELETE** 权限

[例13] 减少角色的权限

```
REVOKE SELECT  
ON TABLE Student  
FROM R1;
```

使**R1**减少了**SELECT**权限



4.2.6 强制存取控制方法（自学）

■ 自主存取控制的缺点是：

➤ 可能存在数据的“无意泄露”。

■ 原因在于：

➤ 这种机制仅仅通过对数据的存取权限来进行安全控制，而数据本身并无安全性标记。

■ 解决方法：

➤ 对系统控制下的所有主客体实施强制存取控制策略。



4.2.6 强制存取控制方法

■ 在MAC中，DBMS所管理的全部实体被分为两大类：

1. 主体 —— 系统中的活动实体，包括：

- ★ DBMS所管理的实际用户
- ★ 代表用户的各进程

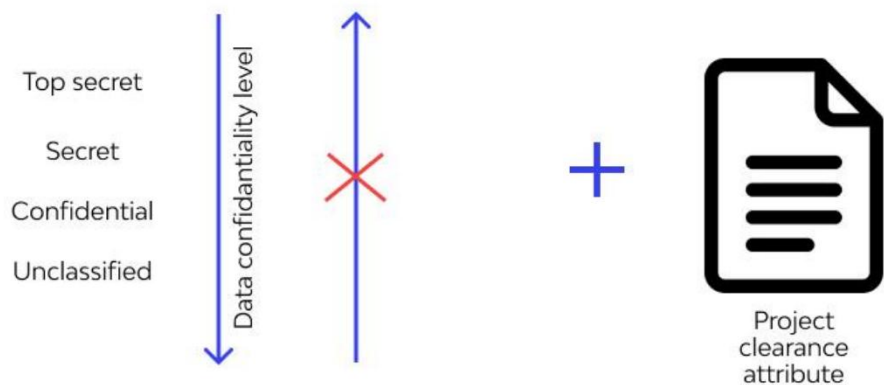
2. 客体 —— 系统中的被动实体，是受主体操纵的，包括：

- ★ 文件
- ★ 基表
- ★ 索引
- ★ 视图 等



4.2.6 强制存取控制方法

Mandatory Access Control (MAC)



图示：MAC模型中数据机密性层级与访问控制逻辑

核心特征：系统强制实施，用户无法自主更改权限或传播数据，是保障机密性与完整性的“铜墙铁壁”。



适用场景：高密级系统

军事、政府及核心机密系统，对应 TCSEC 标准 B1 级及以上安全等级。



核心思想：双向标签化

给数据（客体）打密级标签，给用户（主体）发许可证，严格按标签匹配访问。



访问铁律：读写限制

- 向下读：仅读 $\leq$ 自身级别的数据（防泄密）
- 向上写：仅写 $\geq$ 自身级别的数据（防污染）

强制存取控制方法（续）

■ 强制安全性机制

- 对于主体和客体，DBMS为它们的每个实例（值）指派一个敏感度标记（Label）。敏感度标记被分为若干级别，例如：
 - ★ 绝密（Top Secret）
 - ★ 机密（Secret）
 - ★ 可信（Confidential）
 - ★ 公开（Public） 等
- 主体的敏感度标记称为许可证级别（Clearance Level）；
- 客体的敏感度标记称为密级（Classification Level）。
- MAC机制就是通过对比主体的Label和客体的Label，最终确定主体是否能够存取客体。



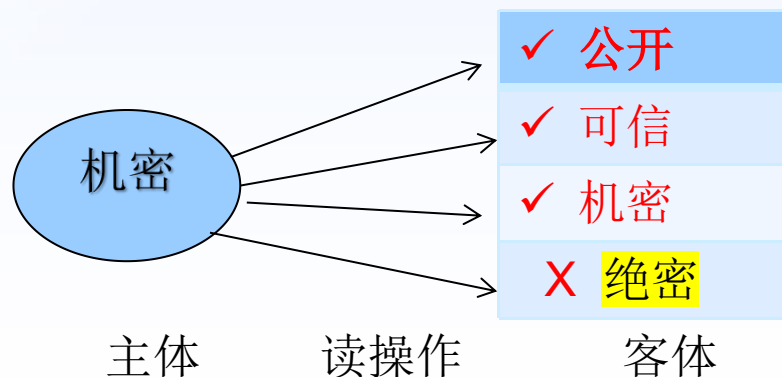
强制存取控制方法（续）

■ 强制存取控制规则：

➤ 当某一用户（或某一主体）以标记Label注册入系统时，系统要求他对任何客体的存取必须遵循下面两条规则：

1. 仅当主体的许可证级别**大于或等于**客体的密级时，该主体才能读取相应的客体。

即： 用户**S**, 不能读取数据对象**O**, 除非 $\text{Level}(S) \geq \text{Level}(O)$



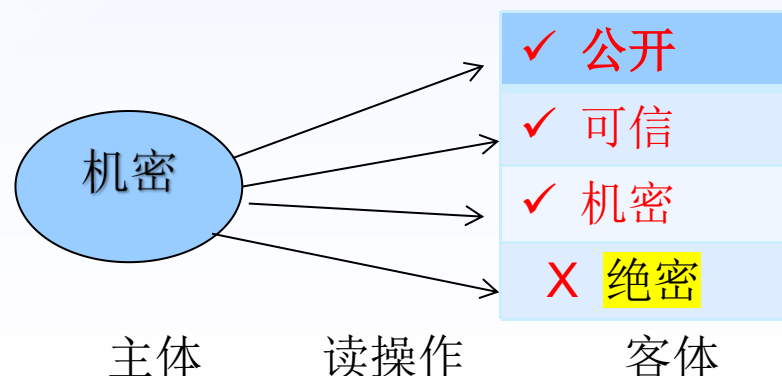
强制存取控制方法（续）

■ 强制存取控制规则：

➤ 当某一用户（或某一主体）以标记Label注册入系统时，系统要求他对任何客体的存取必须遵循下面两条规则：

1. 仅当主体的许可证级别**大于或等于**客体的密级时，该主体才能读取相应的客体。

即： 用户**S**, 不能读取数据对象**O**, 除非 $\text{Level}(S) \geq \text{Level}(O)$

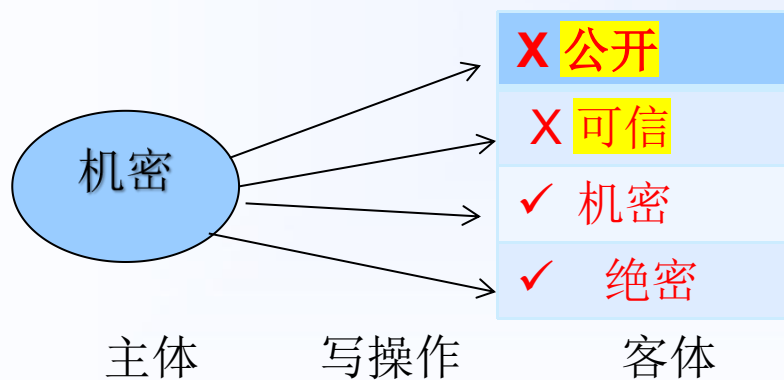


强制存取控制方法（续）

■ 强制存取控制规则：

2. 仅当主体的许可证级别**等于**客体的密级时，该主体才能写相应的客体。

即： 用户**S**, 不能写数据对象**O**, 除非 $\text{Level}(\text{S})=\text{Level}(\text{O})$ 。



强制存取控制方法（续）

■ 某些系统将第（2）条修正为：

2. 仅当主体的许可证级别小于或等于客体的密级时，主体才能写客体。

即： 用户S, 不能写数据对象O, 除非 $\text{Level}(S) \leq \text{Level}(O)$ 。

➤ 也就是说，用户可为写入的数据对象赋予高于自己的许可证级别的密级

➤ 这样，一旦数据被写入，该用户自己也不能再读该数据对象了。

■ 这两种规则的共同点是：禁止了拥有高许可证级别的主体更新低密级的数据对象，从而防止了敏感数据的泄露。



4.2.6 强制存取控制方法

■ 强制存取控制的特点

- **MAC**是对数据本身进行密级标记
- 无论数据如何复制，标记与数据是一个不可分的整体，只有符合密级标记要求的用户才可以操纵数据，从而提供了更高级别的安全性

```
CREATE TABLE Salary-copy  
AS SELECT Eno, Name, Salary  
FROM EMP-Salary;
```

```
Grant SELECT  
ON TABLE Salary-copy  
TO PUBLIC;
```

财务人员A: 绝密

EMP-Salary: 绝密数据

Salary-copy: 绝密数据

许可证级别不是绝密的用户仍然不能访问Salary-copy



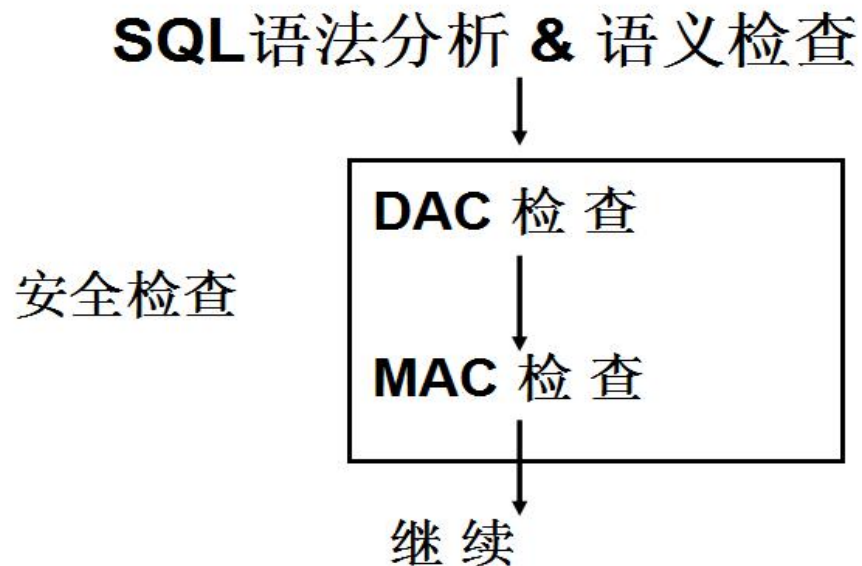
DAC + MAC安全检查

■ DAC与MAC

- 实现强制存取控制MAC时要首先实现自主存取控制DAC，通过DAC检查的数据对象再由系统进行MAC检查，只有通过MAC检查的数据对象方可存取。

原因：较高安全性级别提供的安全保护要包含较低级别的所有保护

- DAC与MAC共同构成DBMS的安全机制



小结

■ 数据库角色

- 创建角色
- 给角色授权
- 将角色授予用户或
- 其他角色
- 角色权限的回收

■ 强制存取控制

- 为主体的每个实例指派一个许可证级别
- 为客体的每个实例指派一个密级
- 通过许可证级别与密级间匹配关系进行存取控制



数据库安全性练习

1. 创建角色**r_oper**，并授予其**connect**权限。
2. 将下列对象权限授予**r_oper**:
 dept表的**select, insert, delete, update**;
 majors表的全部权限; (**all privileges**)
 students, sc, course表的**select**权限。
3. 新建用户**u10**,将角色权限**r_oper**授予用户**u10**。
4. 撤销（收回）用户**u10**的所有权限。
5. 删除用户**u10**。



To be continued
Practice makes perfect!



厚德

笃学

崇实

尚新